

404 Project Not Found

Overview

You're about to build a 2-in-1 web app that combines task management, and image annotation. Basically, the kind of app that makes a recruiter say *“Okay, they get it.”*

You can use any tools, libraries, or AI to help you - Claude code, ChatGPT, Copilot, Tailwind, Figma, and friends are all fair game as long as you can explain the heck you built. Just make sure your implementation is **clean and modular**. You have to use react/nextjs for your frontend and for the backend api use of django is a must (django rest framework is not mandatory for this task but you can use it to make some of the things a lot easier!).

Pages you will be building

0. Please let me into the app! (Login Page)

Well just a simple login interface where user will be able to login using **Email** and **Password**.

1. Nobody does the task (Task page) (/tasks)

Build a Kanban board that lets users manage tasks by date (which will be a separate component). The user should be able to select a date and see (or add) tasks specifically for that day.

Core Features:

- **Columns:** “To Do”, “In Progress”, “Done”
- **Date Picker:** view and manage tasks for the selected day
- **Add/edit/delete** tasks with modals
- **Drag and drop** tasks between columns
- Each task has **title**, **priority**, **due date**, and **tags**

Component Expectations:

- `<DateSelector/>`: shared, reusable date picker component
- `<Board/>`, `<Column/>`, `<TaskCard/>`: modular task structure

- Tasks should be **filtered and shown per date**
- Persist all task data to the database using the backend.

Technical Expectations:

- Implement drag-and-drop
 - Sync date state across components (use useContext, Zustand, or Redux)
 - Clean handling of edge cases (e.g. no tasks for selected date, invalid data)
 - Ensure component separation: date logic should not be tightly coupled with task UI
 - Create proper models to store this data in the backend
-

2. A great annotation ahead! (/annotate)

An annotation tool where the user annotates or you can say draws on images. [Click Here](#) to view this video to get a brief idea on this annotation tool

Core Features:

- Upload images to the backend.
- slide through the uploaded images at once
- Draw or remove drawn shapes from the images.
- Save the drawn shapes into the database using the backend.

Technical Expectations:

- Should be able to scroll multiple images at once.
 - Should be able to draw polygons on images.
 - Remove specific drawn polygon from the image.
 - Use database using backend for persistence
-

Project Instructions

You're free to use whatever tools, libraries, or even AI sidekicks you want — Claude, Copilot, ChatGPT, Figma, Tailwind, django rest framework etc. Summon the full power of your dev arsenal. The goal is to show us not just how you code, but how you **think**, solve problems, and bring ideas to life like a true warrior.

But like any great anime arc, there are a few ground rules:

- You must use **TypeScript** — no JavaScript-only universes here.
- You must use **Django** for your backend and **django ORM** for dealing with queries
- Use your imagination to create a UI that looks great and feels aesthetic. **Bonus points** for being able to create exceptionally **aesthetic design**!
- For databases, as you will be using Django ORM you can just use **sqlite** engine to store the data, but other relational database engines are also welcomed.
- Provide a **readme.md** for both frontend and backend, where you will write your difficulties or the villains you faced throughout the saga, and how you overcame that villain (of course it might be with the power of friendship with a documentation or an AI or a blog!). In another paragraph, include details of the node version, python version and detailed steps to run this project.
- **Push the code into github and host your frontend and backend** in a public hosting site like vercel or netlify or pythonanywhere. Share the links of frontend github repo, backend github repo and hosted app, along with the demo user email and password so that we can login.
- Create a video showcasing your work. The video duration should not exceed more than 2 minutes

Push boundaries. Build with style. Bonus points for clean code, aesthetic UI/UX, and components that don't break under pressure.

"Believe in the code that believes in you!" – not quite Kamina, but close enough. 🗡️🔥

We're not just looking for working apps — we're looking for **developers with main-character energy**.